Async/Sync Communication in .NET C#

Protocols, Patterns & Practices

Agenda

1. Synchronous vs Asynchronous Fundamentals
2. Common Protocols: HTTP, gRPC, Messaging
3. Trade-offs in Scalability & Resilience
4. Case Studies in Distributed Systems
5. Practical Exercises with Microservices
6. AI Agentics for Microservices

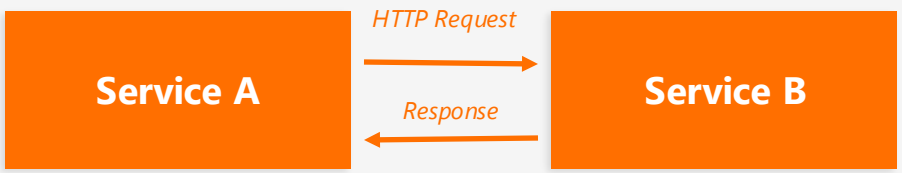
01

Synchronous vs Asynchronous Fundamentals

Section 1 of 7

D01: Sync vs Async Communication Model

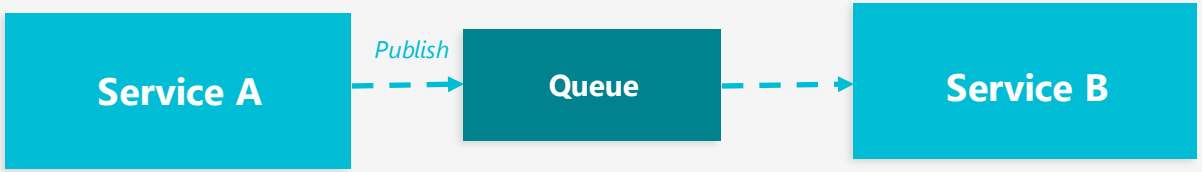
SYNCHRONOUS



Caller BLOCKS
until response

Thread blocked waiting...

ASYNCHRONOUS



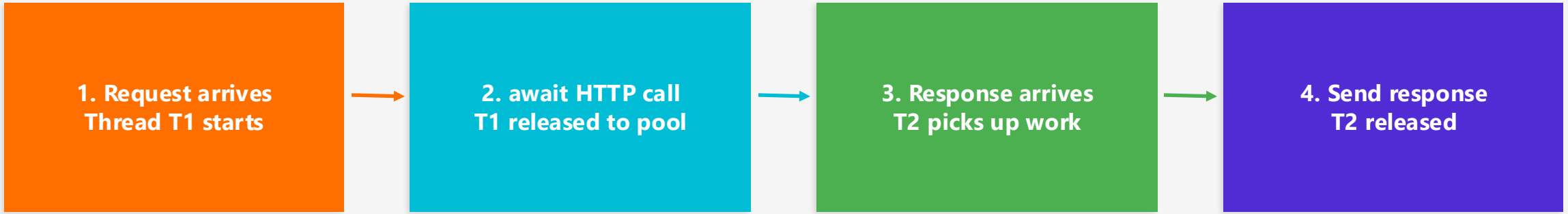
Caller continues
immediately

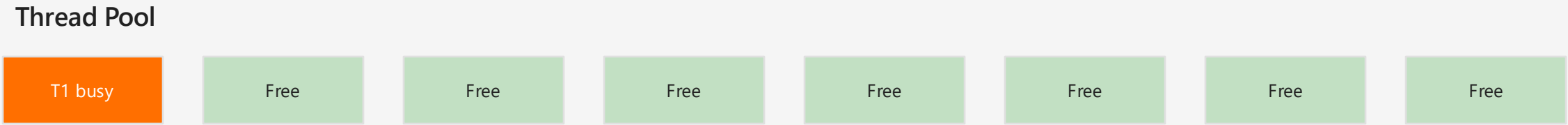
Fire & forget

Synchronous call

Async message

D02: Thread Pool Behavior Under await





With await: only 1 thread busy per I/O step, rest available for other requests

Thread actively processing

Thread free (available)

Language-Level vs Architectural Async

Language-Level (async/await)

- Frees threads during I/O waits
- Caller still waits for a response
- Same request-response pattern
- Improves throughput, not decoupling
- Built into C# compiler

VS

Architectural (Message Queue)

- Producer does not wait for consumer
- Complete temporal decoupling
- Fire-and-forget or eventual reply
- Improves resilience and scalability
- Requires infrastructure (broker)

Sync vs Async: Detailed Comparison

Dimension	Synchronous	Asynchronous
Threading	Blocks thread (or uses await)	Fire-and-forget, no waiting
Coupling	Temporal + spatial coupling	Only spatial coupling (low)
Latency	Total = sum of all hops	Producer returns immediately
Error Handling	Try/catch, HTTP status codes	Dead-letter queues, retries
Complexity	Simple request-response	Needs broker, idempotency
Data Freshness	Always current	Eventually consistent
Scalability	Limited by slowest service	Independently scalable

Asynchronous C# — async/await Pattern

```
// CORRECT: Non-blocking async/await
using var client = new HttpClient();

// await releases the thread back to the pool
var response = await client
    .GetStringAsync("https://api.orders.local/api/orders/123");

// Thread was FREE to serve other requests while waiting
// The continuation runs when the response arrives
Console.WriteLine(response);

// In Minimal API context:
app.MapGet("/orders/{id}", async (string id) =>
{
    var data = await orderClient.GetOrderAsync(id);
    return Results.Ok(data);
});
```

Synchronous C# — The Blocking Anti-Pattern

```
// WARNING: Blocking call — do NOT use in production
using var client = new HttpClient();


// .Result blocks the calling thread until completion
var response = client
    .GetStringAsync("https://api.orders.local/api/orders/123")
    .Result;


// Thread was BLOCKED during the entire HTTP round-trip
// In ASP.NET, this can cause thread pool starvation
Console.WriteLine(response);
```

Key Takeaways

1

async/await frees threads, not callers

Language-level async optimizes thread usage but maintains request-response coupling.

2

Architectural async decouples services

Message queues eliminate temporal coupling — producer and consumer run independently.

!

Never call .Result in ASP.NET

Blocking the thread pool causes starvation and cascading failures under load.

3

Use both together

async/await within services + message queues between services = best scalability.

02

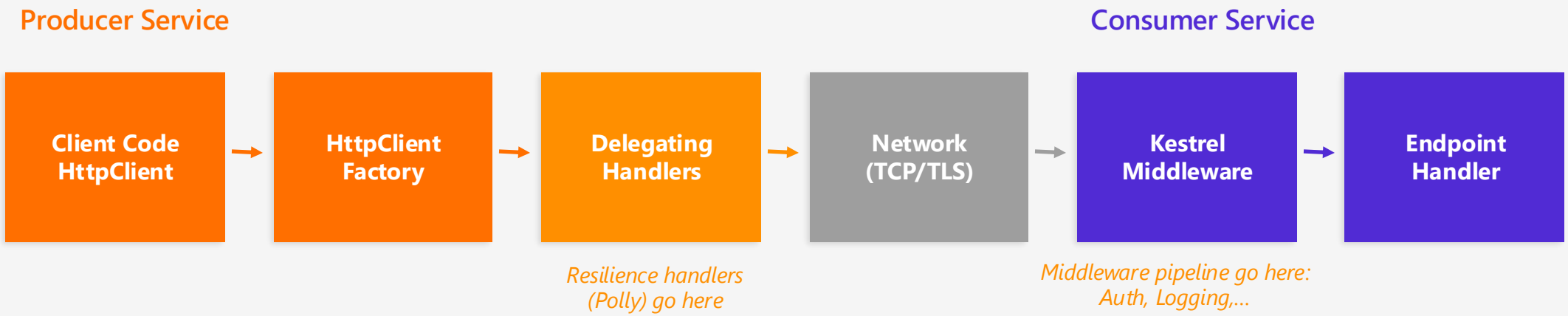
Common Protocols: HTTP, gRPC, Messaging

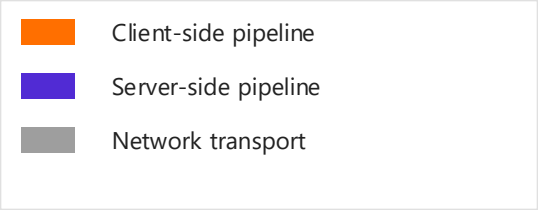
Section 2 of 7

D03: Protocol Landscape — HTTP / gRPC / Messaging

HTTP / REST	gRPC	Message Queues
Text-based (JSON)	Binary (Protobuf)	Any format
Universal support	Code generation	Decoupled services
Browser-friendly	Streaming support	Buffered delivery
Req/Response	Req/Response + Stream	Pub/Sub + P2P
Latency: Medium	Latency: Low	Latency: Variable

D04: HTTP Request Flow in .NET





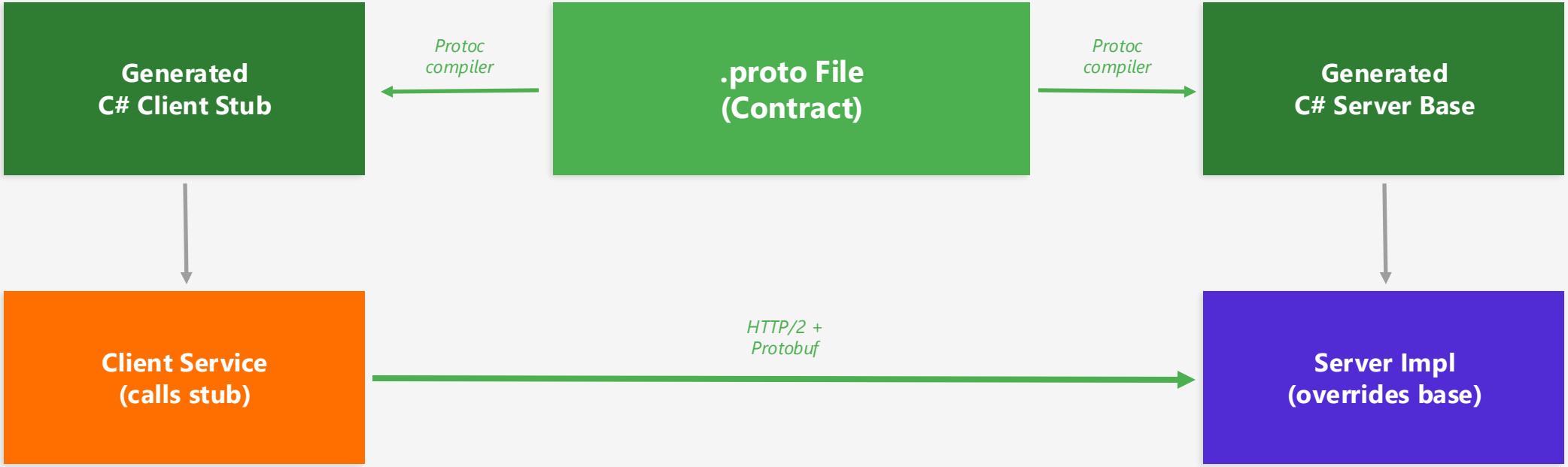
Minimal API Endpoint (MapGet)

```
// Program.cs — .NET 8 Minimal API
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<OrderRepository>();

var app = builder.Build();

app.MapGet("/api/orders/{id}", async (
    string id,
    OrderRepository repo) =>
{
    var order = await repo.GetByIdAsync(id);
    return order is not null
        ? Results.Ok(order)
        : Results.NotFound();
});

app.Run();
```

IHttpClientFactory — Consuming HTTP APIs

```
// In your service registration:
builder.Services.AddHttpClient("OrdersApi", client =>
{
    client.BaseAddress = new Uri("https://orders-api:5001");
    client.DefaultRequestHeaders.Add("Accept", "application/json");
})
.AddStandardResilienceHandler(); // Polly resilience

// In your handler or service class:
public class OrderService(IHttpClientFactory factory)
{
    public async Task<Order?> GetOrderAsync(string id)
    {
        var client = factory.CreateClient("OrdersApi");
        var response = await client.GetAsync($"/api/orders/{id}");
        response.EnsureSuccessStatusCode();
        return await response.Content
            .ReadFromJsonAsync<Order>();
    }
}
```

D05: gRPC Architecture (Proto to Code)



> Binary serialization (10x faster)

> HTTP/2 multiplexing

> Streaming (server, client, bidirectional)

> Strong typing from .proto contracts

- gRPC / Protobuf
- Client impl
- Server impl

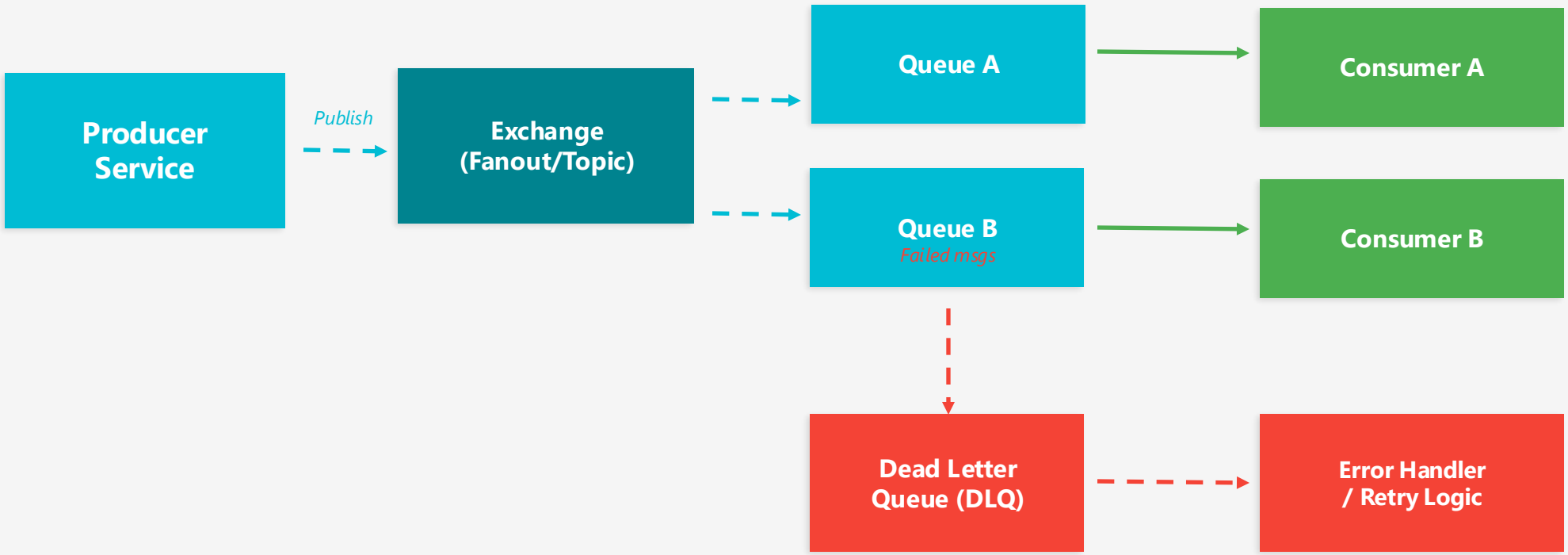
.proto File — gRPC Contract Definition

```
syntax = "proto3";

option csharp_namespace = "OrderService.Grpc";

package orders;

service OrderGrpc {
  rpc GetOrder (OrderRequest) returns (OrderReply);
  rpc StreamOrders (OrderFilter) returns (stream OrderReply);
}

message OrderRequest {
  string order_id = 1;
}

message OrderFilter {
  string status = 1;
}

message OrderReply {
  string order_id = 1;
  string product = 2;
  int32 quantity = 3;
  double total_price = 4;
}
```

gRPC Server & Client in C#

```
// SERVER: Implement the generated base class
public class OrderGrpcService : OrderGrpc.OrderGrpcBase
{
  public override async Task<OrderReply> GetOrder(
    OrderRequest request, ServerCallContext context)
  {
    var order = await _repo.GetByIdAsync(request.OrderId);
    return new OrderReply
    {
      OrderId = order.Id,
      Product = order.Product,
      Quantity = order.Qty,
      TotalPrice = order.Total
    };
  }
}

// CLIENT: Use the generated typed client
var channel = GrpcChannel.ForAddress("https://orders:5001");
var client = new OrderGrpc.OrderGrpcClient(channel);
var reply = await client.GetOrderAsync(
  new OrderRequest { OrderId = "ORD-123" });
```

D06: Message Queue Topology + Dead Letter Queue



- Async message flow
- Consumption
- Error / DLQ path

MassTransit Producer (IBus.Publish)

```
// Message contract (shared assembly)
public record OrderCreated(
    string OrderId,
    string Product,
    int Quantity,
    decimal Total,
    DateTime CreatedAt);

// Registration in Program.cs
builder.Services.AddMassTransit(x =>
{
    x.UsingRabbitMq((ctx, cfg) =>
    {
        cfg.Host("rabbitmq://localhost");
        cfg.ConfigureEndpoints(ctx);
    });
});

// Publishing from a Minimal API endpoint
app.MapPost("/api/orders", async (
    CreateOrderRequest req, IBus bus) =>
{
    var order = ProcessOrder(req);
    await bus.Publish(new OrderCreated(
        order.Id, order.Product, order.Qty,
        order.Total, DateTime.UtcNow));
    return Results.Accepted();
});
```

MassTransit Consumer (IConsumer<T>)

```
// Consumer implementation
public class OrderCreatedConsumer : IConsumer<OrderCreated>
{
    private readonly ILogger<OrderCreatedConsumer> _log;
    private readonly InventoryService _inventory;

    public OrderCreatedConsumer(
        ILogger<OrderCreatedConsumer> log,
        InventoryService inventory)
    {
        _log = log;
        _inventory = inventory;
    }

    public async Task Consume(ConsumeContext<OrderCreated> ctx)
    {
        var msg = ctx.Message;
        _log.LogInformation(
            "Processing order {Id} for {Product}",
            msg.OrderId, msg.Product);

        await _inventory.ReserveStockAsync(
            msg.Product, msg.Quantity);
    }
}
```

Protocol Comparison Matrix

Aspect	HTTP / REST	gRPC	Message Queue
Pattern	Request-Response	Req/Resp + Streaming	Pub/Sub, P2P
Coupling	Spatial + Temporal	Spatial + Temporal	Spatial only
Payload	JSON (text)	Protobuf (binary)	Any format
Performance	Good	Excellent (10x)	Variable
Browser Support	Full	Limited (gRPC-Web)	None (indirect)
Contract	OpenAPI / Swagger	.proto (strict)	Schema registry
Best For	Public APIs, CRUD	Internal high-perf	Async workflows

Key Takeaways

H

HTTP for universal access

REST/JSON is the default for public APIs and browser-facing services.

G

gRPC for internal speed

Binary Protobuf + HTTP/2 delivers 10x performance for service-to-service calls.

Q

Message queues for resilience

Decouple services temporally using RabbitMQ/MassTransit for async workflows.

*

Use all three strategically

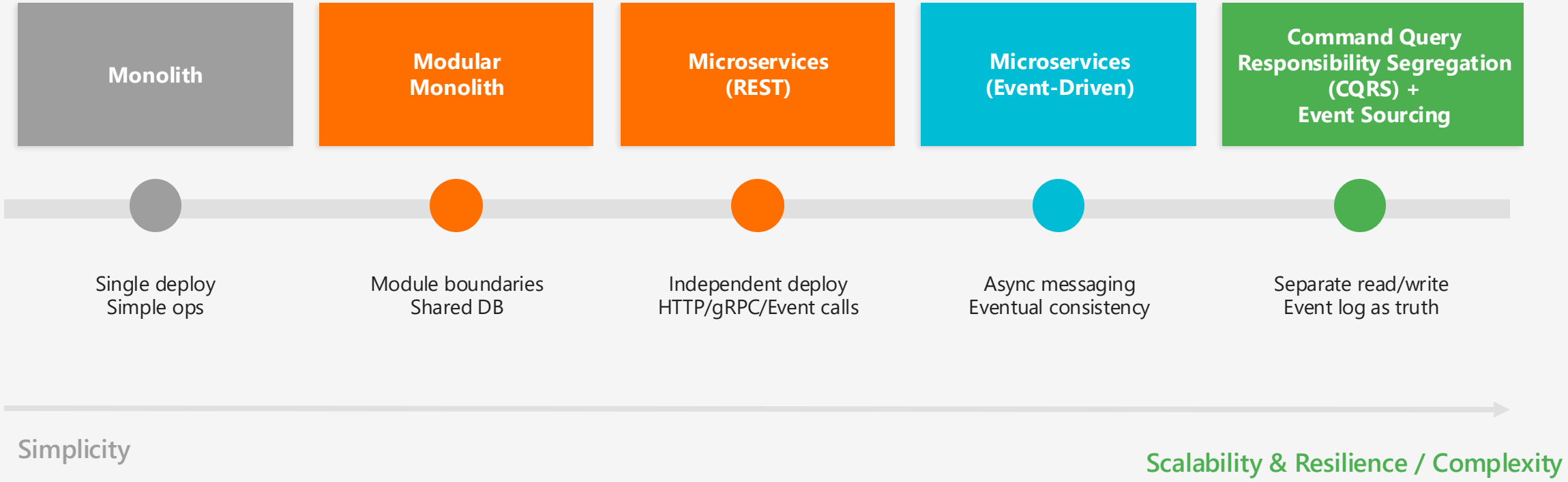
Mature architectures combine HTTP (external), gRPC (internal), queues (async).

03

Trade-offs in Scalability & Resilience

Section 3 of 7

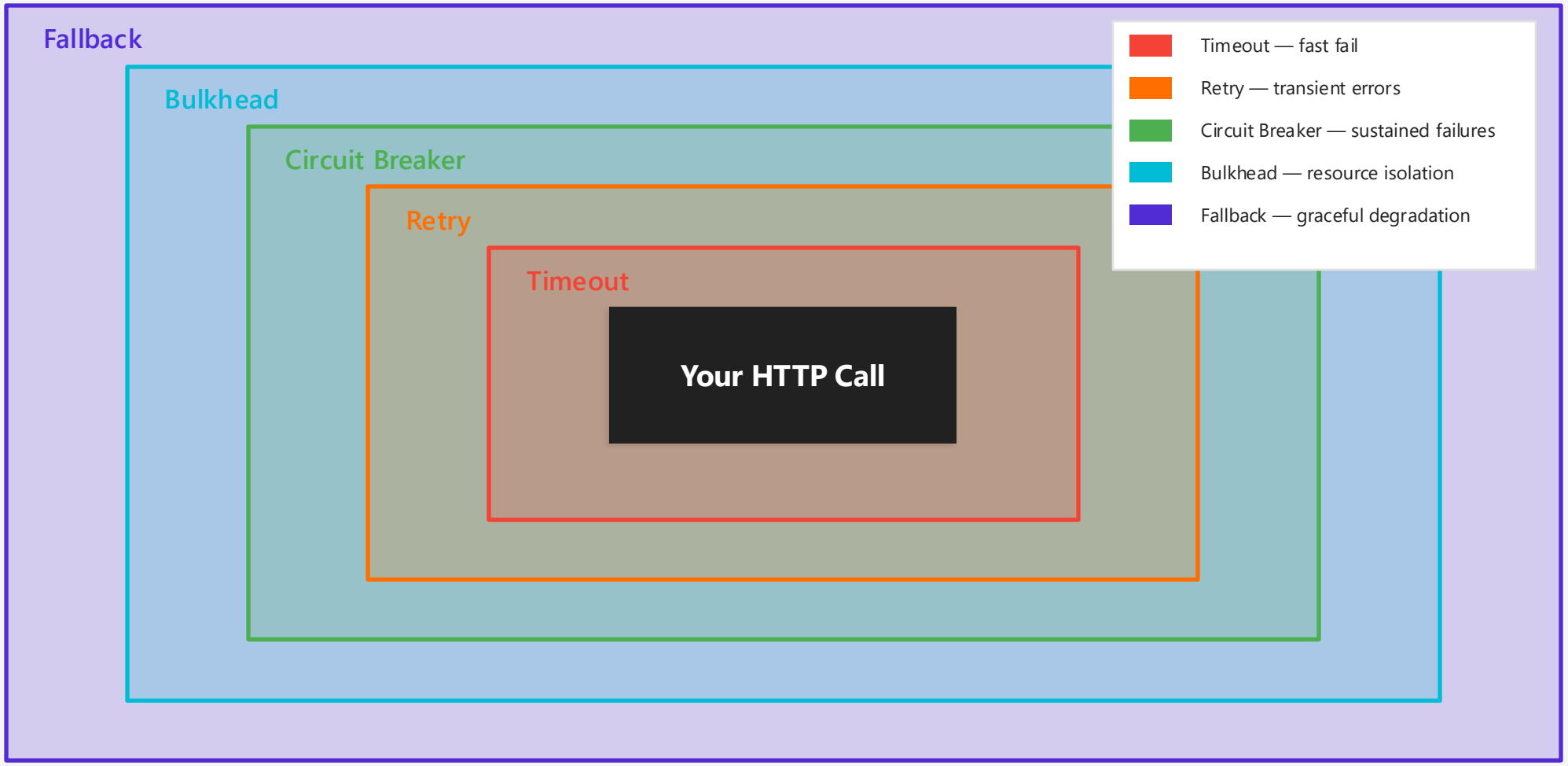
D09: Scalability Spectrum



Scalability Strategies and Their Costs

Strategy	Benefit	Cost / Trade-off
Horizontal scaling	Linear throughput increase	Stateless services required
Database read replicas	Read throughput scales	Replication lag, eventual consistency
Caching (Redis)	Sub-ms response times	Cache invalidation complexity
CQRS	Independent read/write scaling	Data sync complexity, dual models, eventual consistency
Event sourcing	Complete audit trail, replay	Storage growth, projection rebuilds
Sharding	Horizontal data partitioning	Cross-shard queries expensive

D07: Resilience Patterns — Defense in Depth



Outer layers protect inner layers — each pattern handles a different failure mode

Standard Resilience Handler (.NET 8)

```
// One line adds a complete resilience pipeline!
builder.Services.AddHttpClient("OrdersApi")
    .AddStandardResilienceHandler();

// Or customize each layer:
builder.Services.AddHttpClient("PaymentsApi")
    .AddResilienceHandler("custom", builder =>
    {
        builder
            .AddTimeout(TimeSpan.FromSeconds(5))
            .AddRetry(new HttpRetryStrategyOptions
            {
                MaxRetryAttempts = 3,
                BackoffType = DelayBackoffType.Exponential,
                UseJitter = true
            })
            .AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions
            {
                SamplingDuration = TimeSpan.FromSeconds(10),
                FailureRatio = 0.9,
                MinimumThroughput = 5
            })
            .AddTimeout(TimeSpan.FromSeconds(30));
    });
```

Core Resilience Patterns

R **Retry**

Repeat failed calls with backoff

C **Circuit Breaker**

Stop calling failing services

T **Timeout**

Fail fast on slow responses

B **Bulkhead**

Isolate resources per service

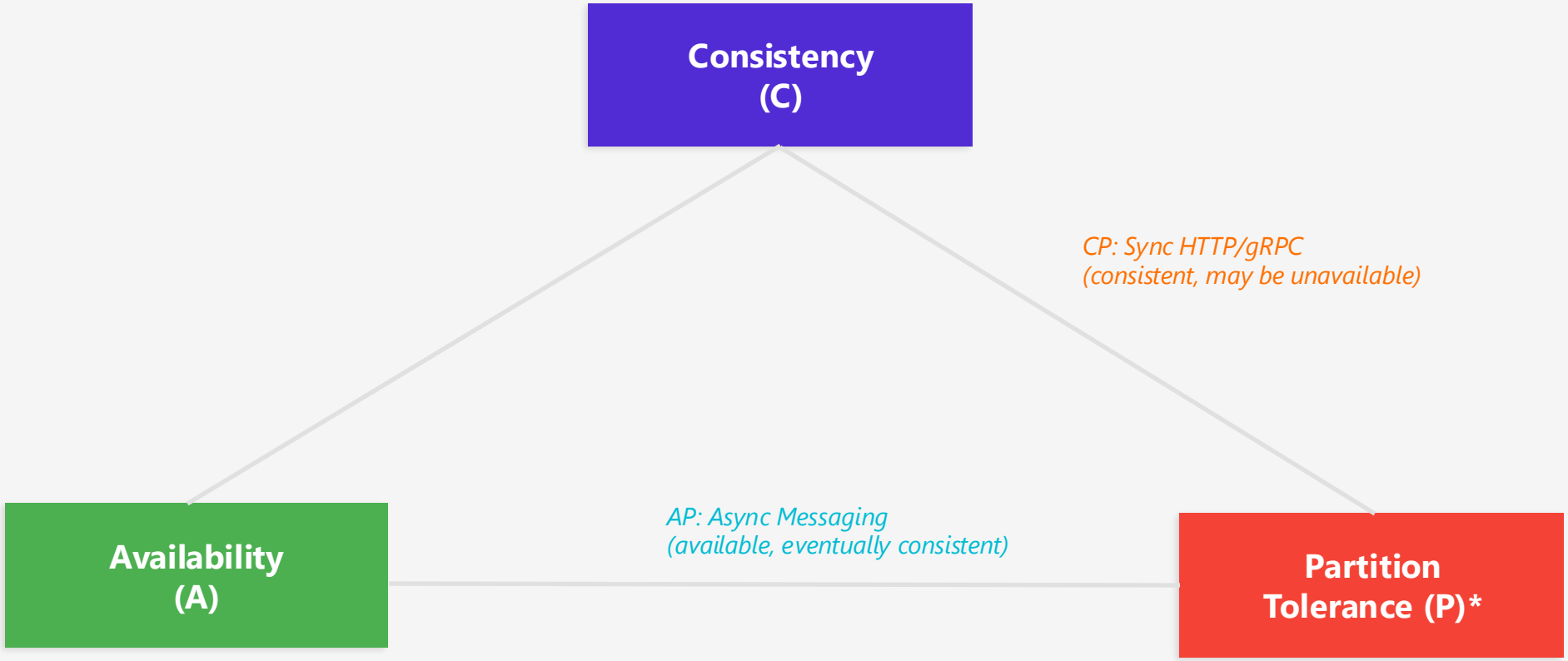
F **Fallback**

Return cached/default on failure

S **Saga**

Coordinate distributed transactions – MassTransit

D08: CAP Theorem — Where Sync & Async Falls



You must choose 2 of 3 in distributed systems

Key Takeaways

1. **Progress along the scalability spectrum**

   Move from monolith to event sourcing only as requirements demand — each step adds complexity.

2. **Layer resilience patterns**

   Timeout + Retry + Circuit Breaker + Bulkhead + Fallback create defense in depth.

3. **CAP is a per-operation decision**

   Choose CP (consistency) or AP (availability) based on each specific business operation.

4. **Use .NET 8 Standard Resilience**

   AddStandardResilienceHandler() provides production-grade defaults in one line.

04

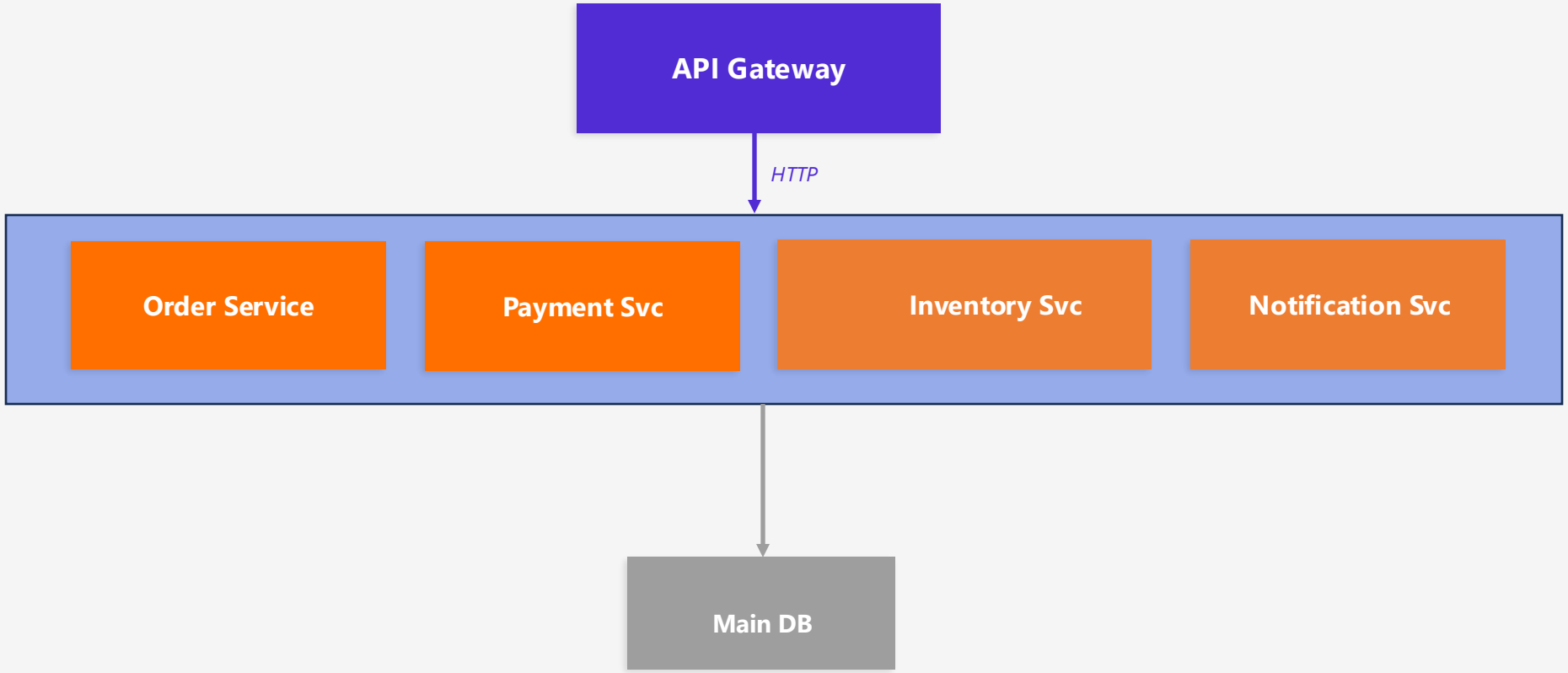
Case Studies in Distributed Systems

Section 4 of 7

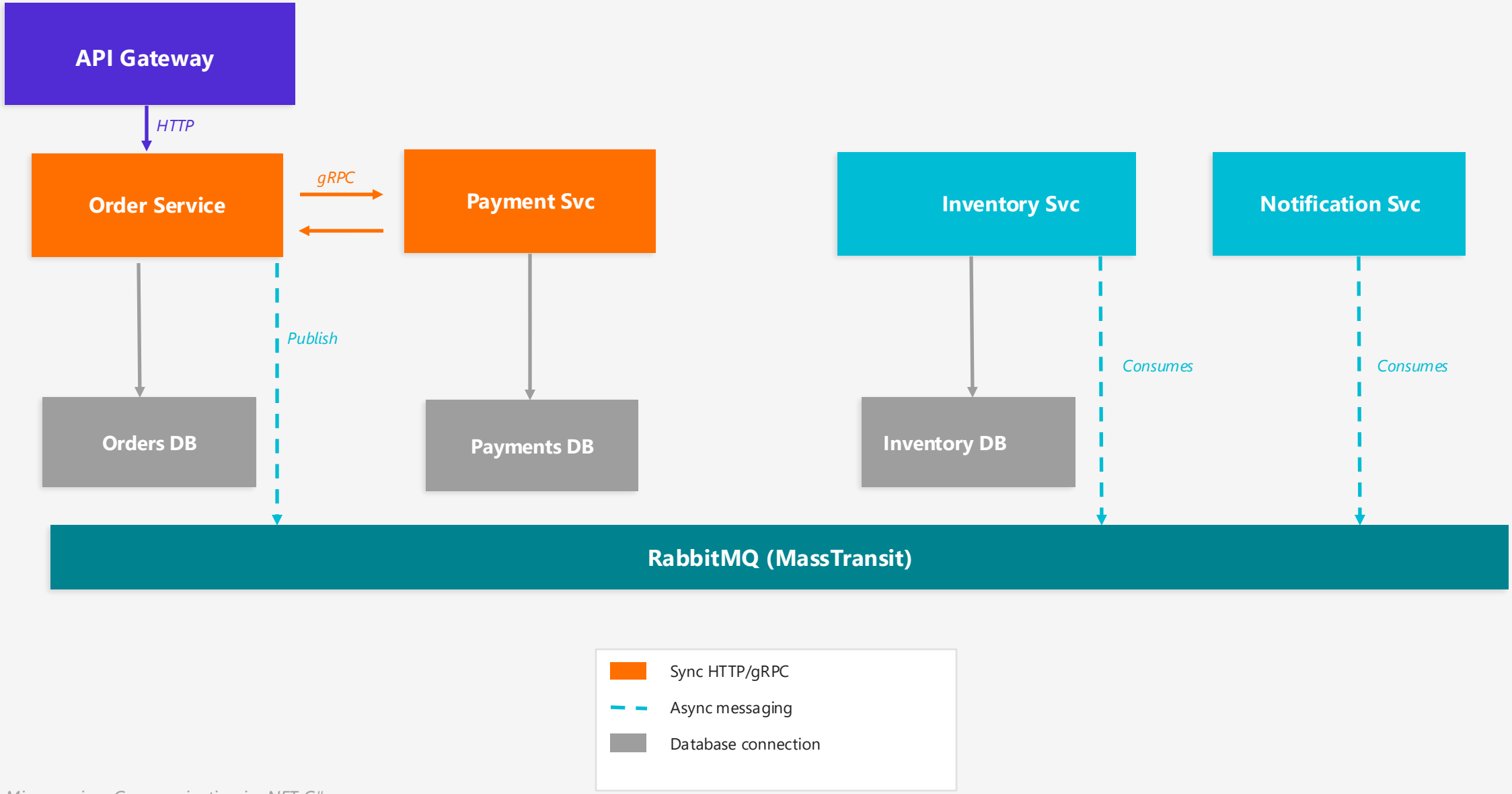
Case Study 1: E-Commerce Order Pipeline — Context

- Online retailer processing 10,000+ orders per hour
- Original: monolithic .NET Framework application
- Pain: payment failures caused full checkout failures
- Pain: inventory service overload during flash sales
- Goal: resilient, independently scalable order pipeline
- Constraint: zero-downtime migration required

D10: E-Commerce Order - Monolith Architecture



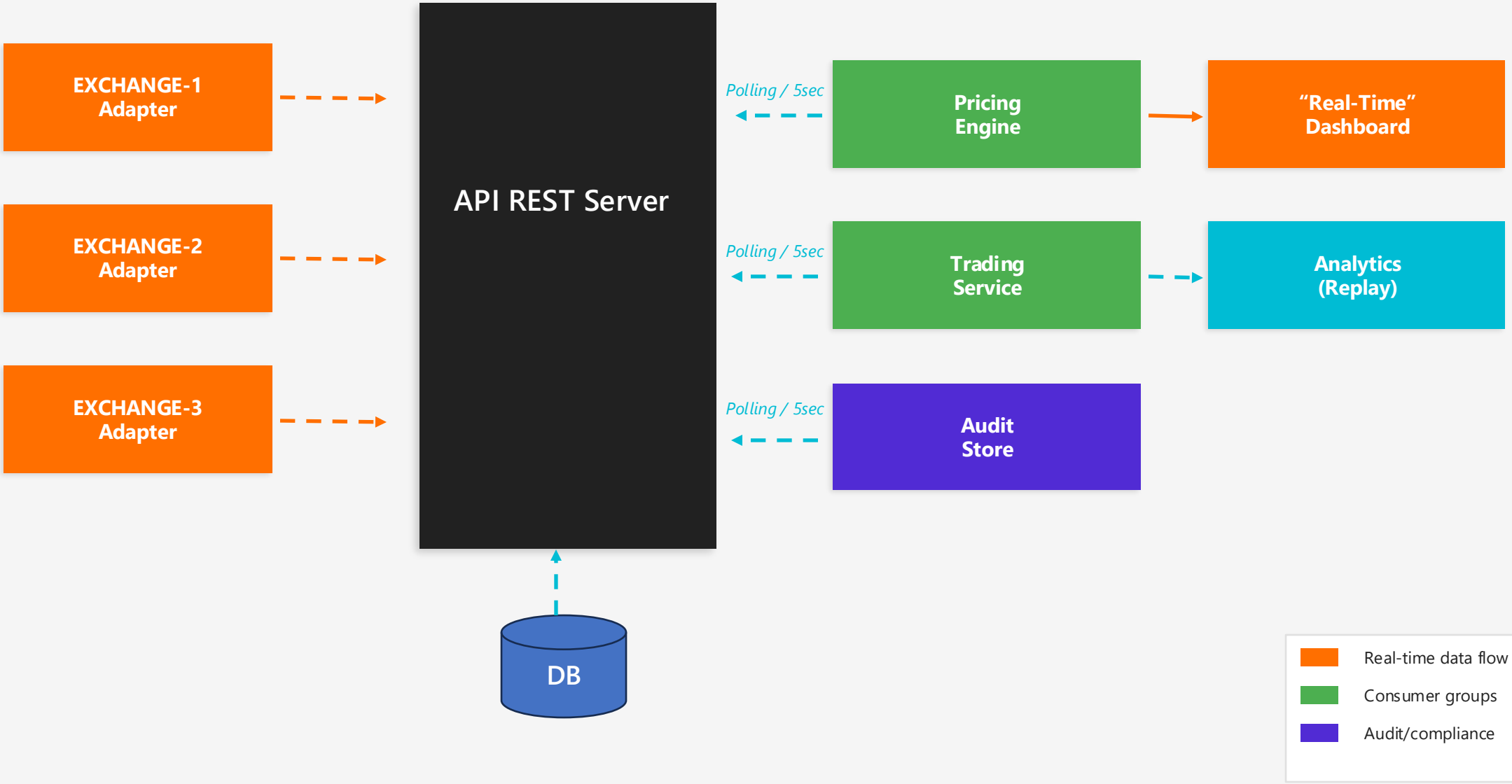
D10: E-Commerce Order - Pipeline Architecture



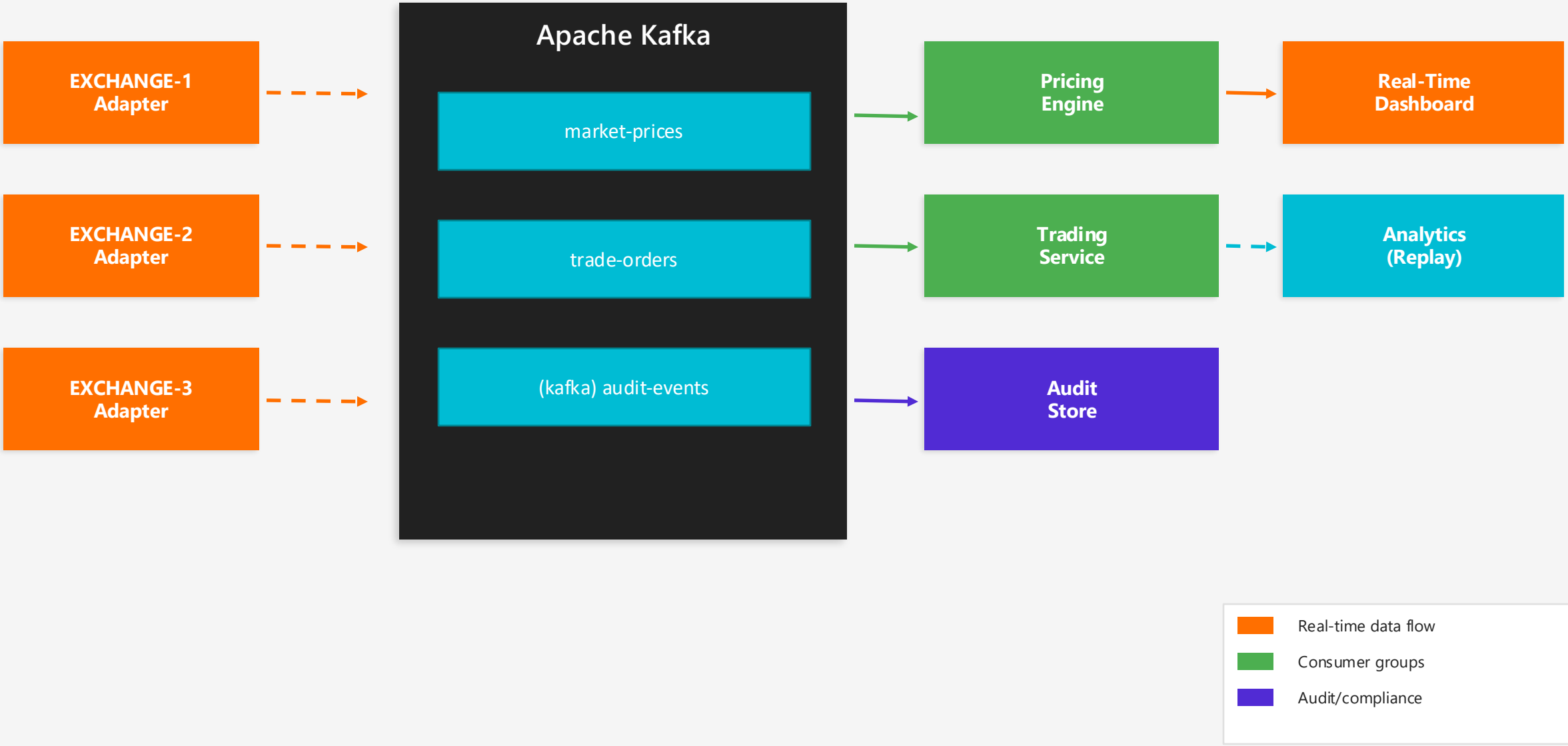
Case Study 2: Real-Time Financial Platform — Context

- Fintech processing market data from 15 exchanges
- Requirement: sub-second latency for price updates
- Volume: 500,000+ events per second during market hours
- Original: polling-based REST architecture (5s delay)
- Goal: real-time event streaming with replay capability
- Constraint: regulatory audit trail requirements

D11: Polling-based REST Financial Data Platform



D11: Real-Time Financial Data Platform (Kafka)



05

Practical Exercises with Microservices

Section 5 of 7

Exercise 1: HTTP Microservice Pair + Resilience

Objective

Build two .NET 8 Minimal API services that communicate via HTTP with full Polly resilience policies.

Steps

- Create OrdersApi with POST/GET endpoints
- Create InventoryApi that OrdersApi calls
- Register IHttpClientFactory with named client
- Add AddStandardResilienceHandler()
- Test with Inventory down to verify circuit breaker

Expected Outcome

Orders API returns cached response when Inventory is unavailable; circuit breaker opens after 5 failures.

Stretch Goal

Implement custom retry strategy with exponential backoff and jitter; add a bulkhead limiting concurrent inventory calls to 10.

Exercise 3: Async Messaging with RabbitMQ + MassTransit

Objective

Introduce asynchronous communication via RabbitMQ and MassTransit for order processing.

Steps

- Run RabbitMQ via Docker Compose
- Add MassTransit NuGet packages
- Define OrderCreated message contract
- Publish events from OrdersApi
- Implement consumer in InventoryApi

Expected Outcome

Orders API returns 202 Accepted immediately; InventoryApi processes messages from the queue asynchronously.

Stretch Goal

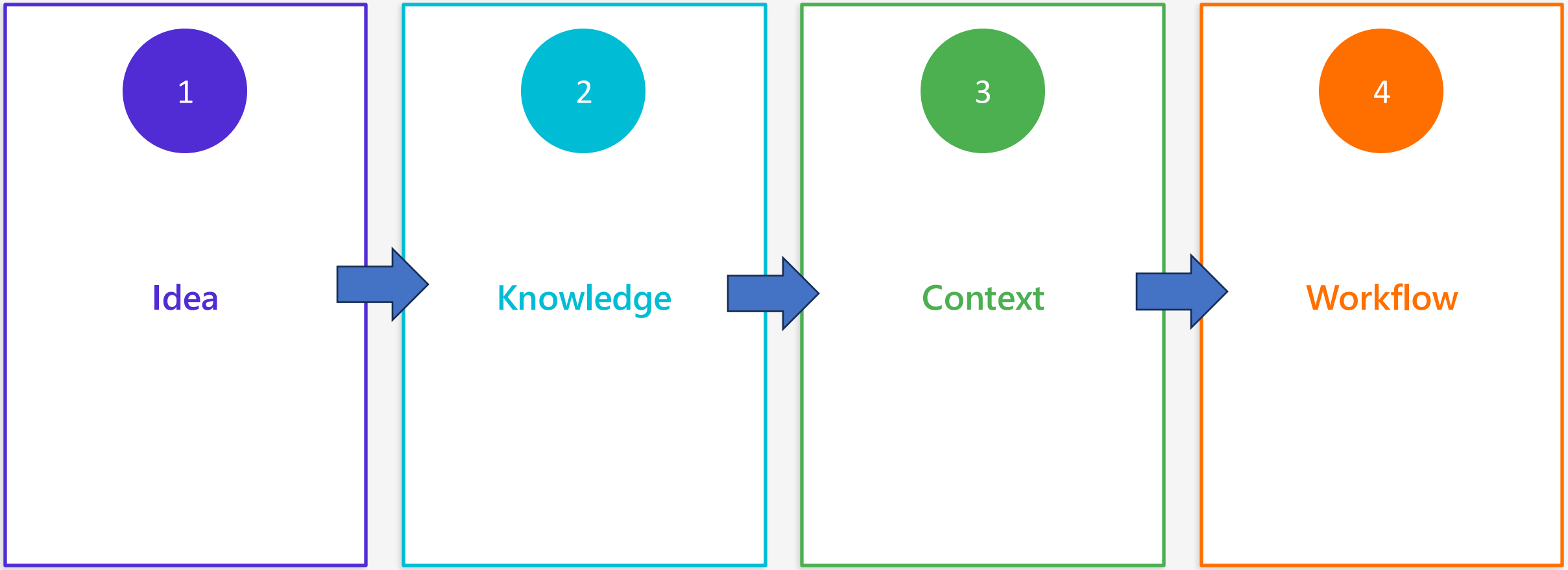
Add a Dead Letter Queue handler; implement idempotent message processing using message deduplication.

06

AI Agentics for Microservices

Section 7 of 7

AI for Microservices



Thank You!

Async/Sync Communication in .NET C#